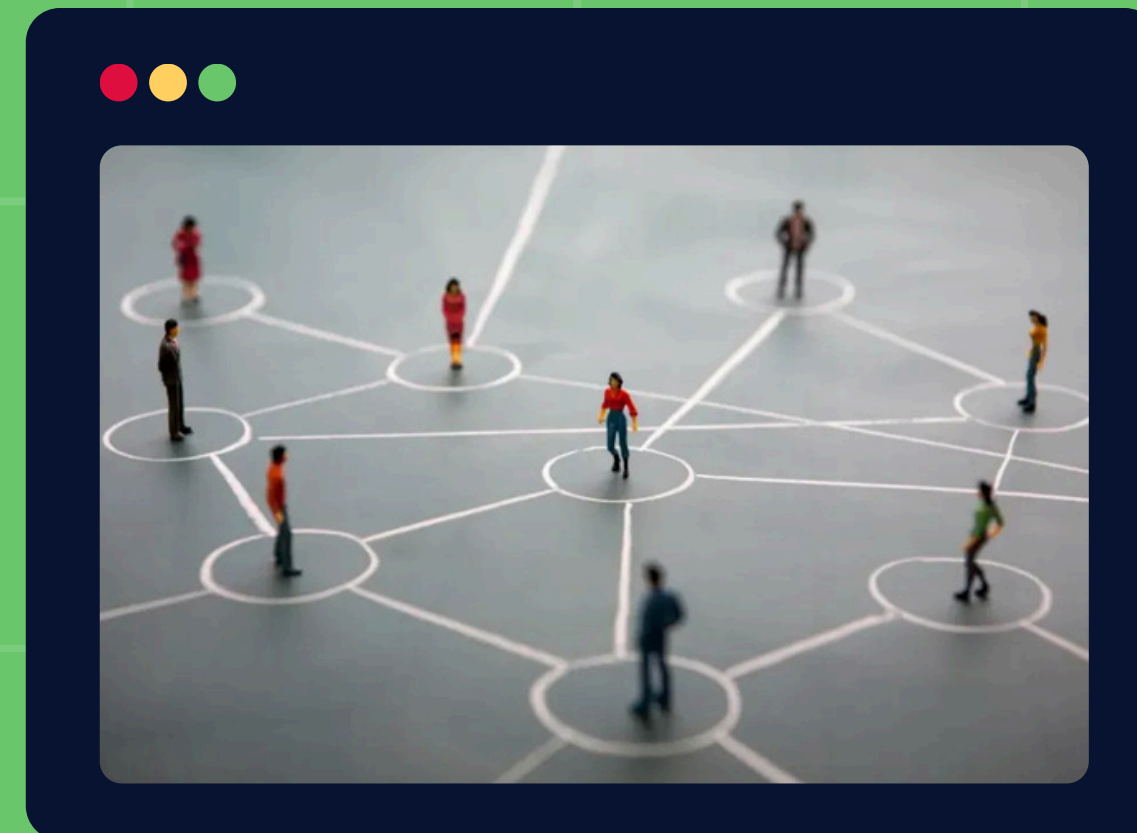


Recomendação de Artigos

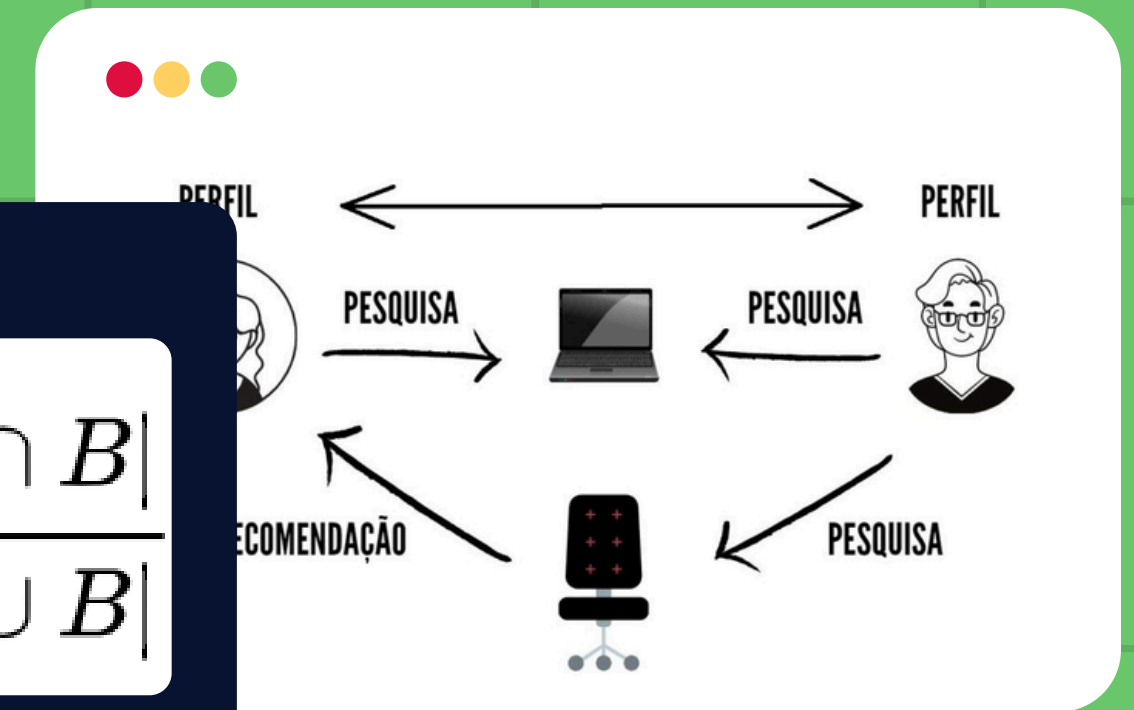


Presented By

GRUPO 14

Definição do Problema

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$



O crescimento exponencial de publicações científicas dificulta que pesquisadores encontrem artigos relevantes de forma eficiente. Sistemas de recomendação surgem como solução para filtrar automaticamente o conteúdo mais adequado ao perfil de cada usuário.

Este trabalho desenvolve um sistema de recomendação de artigos acadêmicos de Ciência da Computação, modelando as interações entre usuários e artigos como um grafo bipartido e calculando a similaridade entre publicações por meio da Similaridade de Jaccard.

Modelagem do Grafo

As interações entre usuários e artigos foram modeladas como um grafo bipartido, onde cada aresta representa uma leitura realizada. A estrutura foi implementada com dois dicionários Python – um mapeando usuários para seus artigos e outro mapeando artigos para seus usuários – garantindo acesso em tempo $O(1)$.

A partir do grafo bipartido, foi construída uma projeção artigo-artigo utilizando a Similaridade de Jaccard, que mede a proporção de leitores em comum entre dois artigos. Arestas com similaridade abaixo de um limiar mínimo são descartadas, eliminando relações fracas e reduzindo o ruído nas recomendações.

```
class GrafoBipartido:
    def __init__(self):
        self._grafo_usuarios = {}
        self._grafo_artigos = {}

    def adicionar_interacao(self, usuario_id, artigo_id):
        #Adiciona uma única aresta ao grafo bipartido.

        self._grafo_usuarios.setdefault(usuario_id, set()).add(artigo_id)
        self._grafo_artigos.setdefault(artigo_id, set()).add(usuario_id)

    def carregar_interacoes(self, interacoes):
        #Popula o grafo iterando sobre uma lista de tuplas (usuario_id, artigo_id).

        for usuario, artigo in interacoes:
            self.adicionar_interacao(usuario, artigo)

    def obter_artigos_do_usuario(self, usuario_id):
```

```
def _jaccard(self, grafo: GrafoBipartido, artigo_a: str, artigo_b: str) -> float:
    """
    Calcula a Similaridade de Jaccard entre dois artigos.

    
$$J(A, B) = |U(A) \cap U(B)| / |U(A) \cup U(B)|$$


    Retorna:
        float: valor entre 0.0 e 1.0. Retorna 0.0 se a união for vazia.
    """
    usuarios_a = grafo.obter_usuarios_do_artigo(artigo_a)
    usuarios_b = grafo.obter_usuarios_do_artigo(artigo_b)

    intersecao = len(usuarios_a & usuarios_b)
    uniao = len(usuarios_a | usuarios_b)

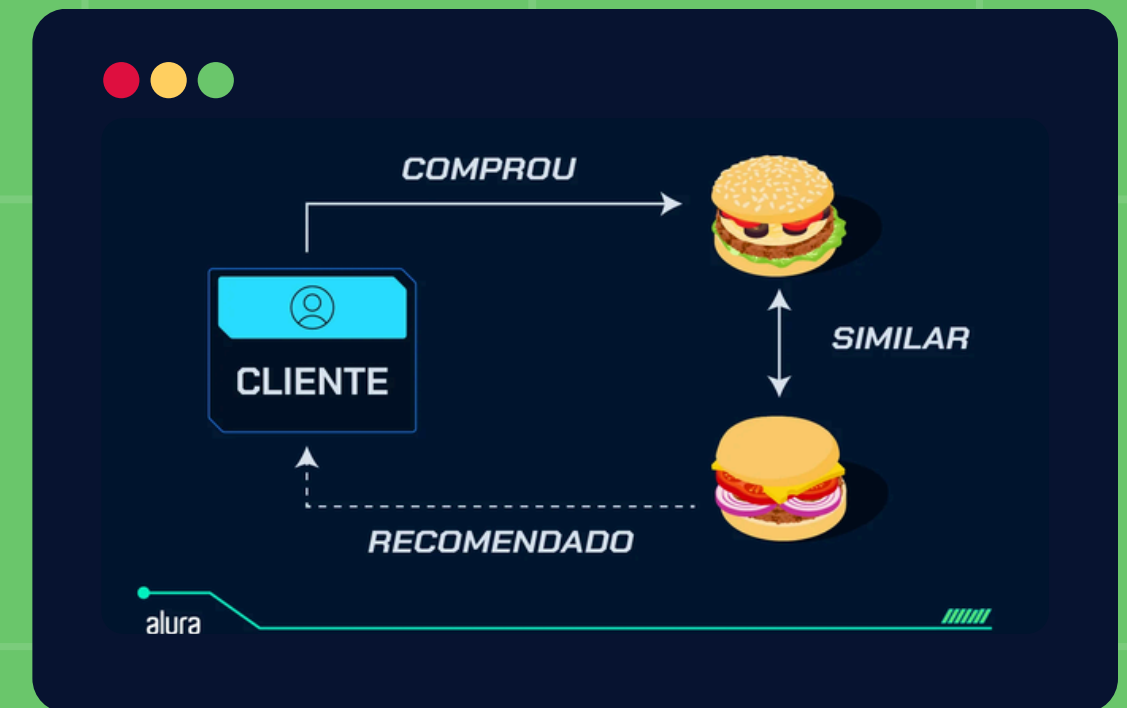
    if uniao == 0:
        return 0.0

    return intersecao / uniao
```

Pipeline da Solução



A solução é composta por um pipeline de cinco etapas: carregamento dos dados em CSV, construção do grafo bipartido, cálculo da projeção artigo-artigo via Jaccard, filtragem das arestas por limiar mínimo e, por fim, geração das recomendações por vizinhos mais próximos.



O sistema oferece dois modos de uso: para usuários existentes, parte-se do histórico de leituras já registrado no grafo; para usuários novos, o usuário informa sua área de interesse e o sistema utiliza os artigos daquela área como base para buscar recomendações na projeção artigo-artigo.

Implementação



O sistema é dividido em quatro módulos Python: **utils.py** carrega os dados dos CSVs para dicionários com acesso $O(1)$; **GrafoBipartido.py** constrói a estrutura central do sistema, dois dicionários que mapeiam usuários para seus artigos e artigos para seus usuários; **projecao.py** gera o grafo artigo-artigo calculando a Similaridade de Jaccard entre pares e descartando arestas fracas via threshold; e **recomendacao.py** consome esse grafo para ranquear sugestões usando uma heap.

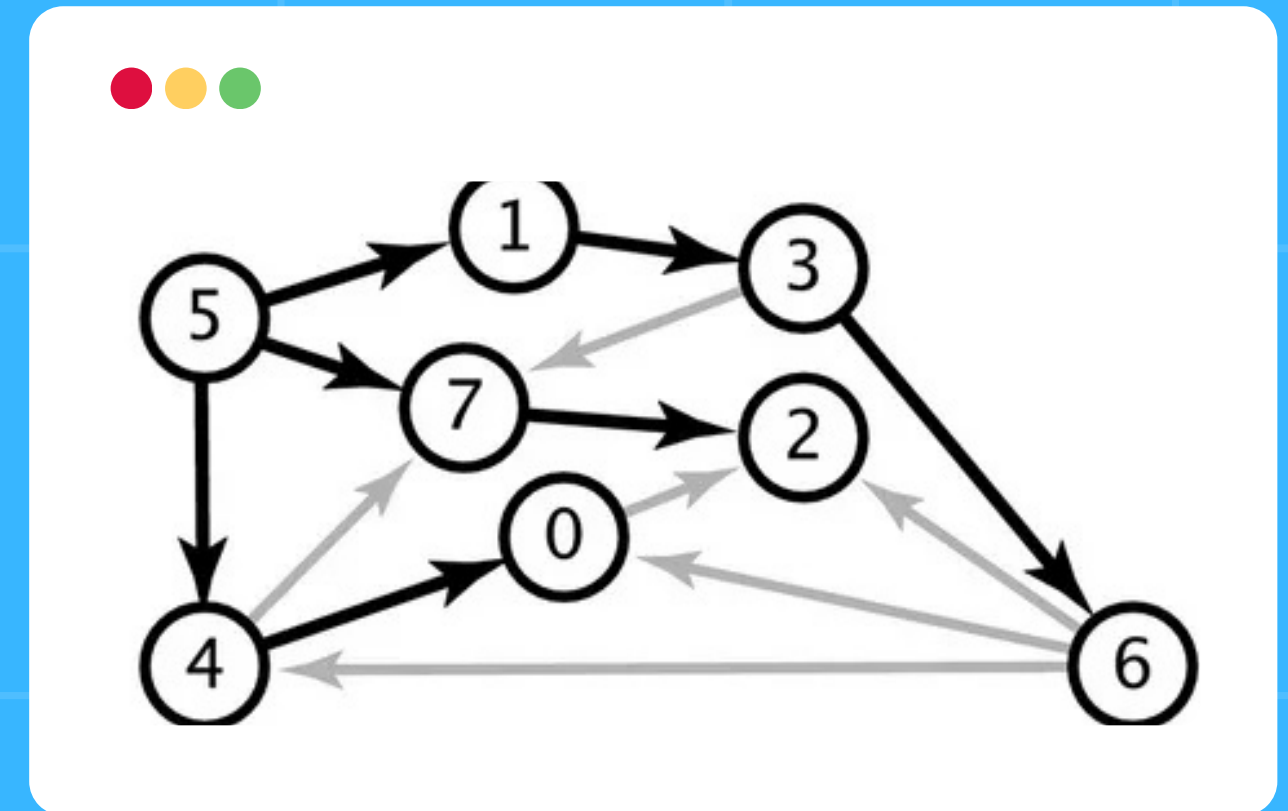


Cada módulo foi implementado como uma classe, seguindo o mesmo padrão: métodos privados cuidam da lógica interna e métodos públicos expõem apenas o necessário. O **GrafoBipartido** registra interações com **adicionar_interacao()** e consulta vizinhos com **obter_artigos_do_usuario()** e **obter_usuarios_do_artigo()**. A **ProjecaoArtigo** encapsula todo o processo de construção e filtragem no `__init__`, expondo apenas **obter_vizinhos_ordenados()** para a camada de recomendação.

Implementação

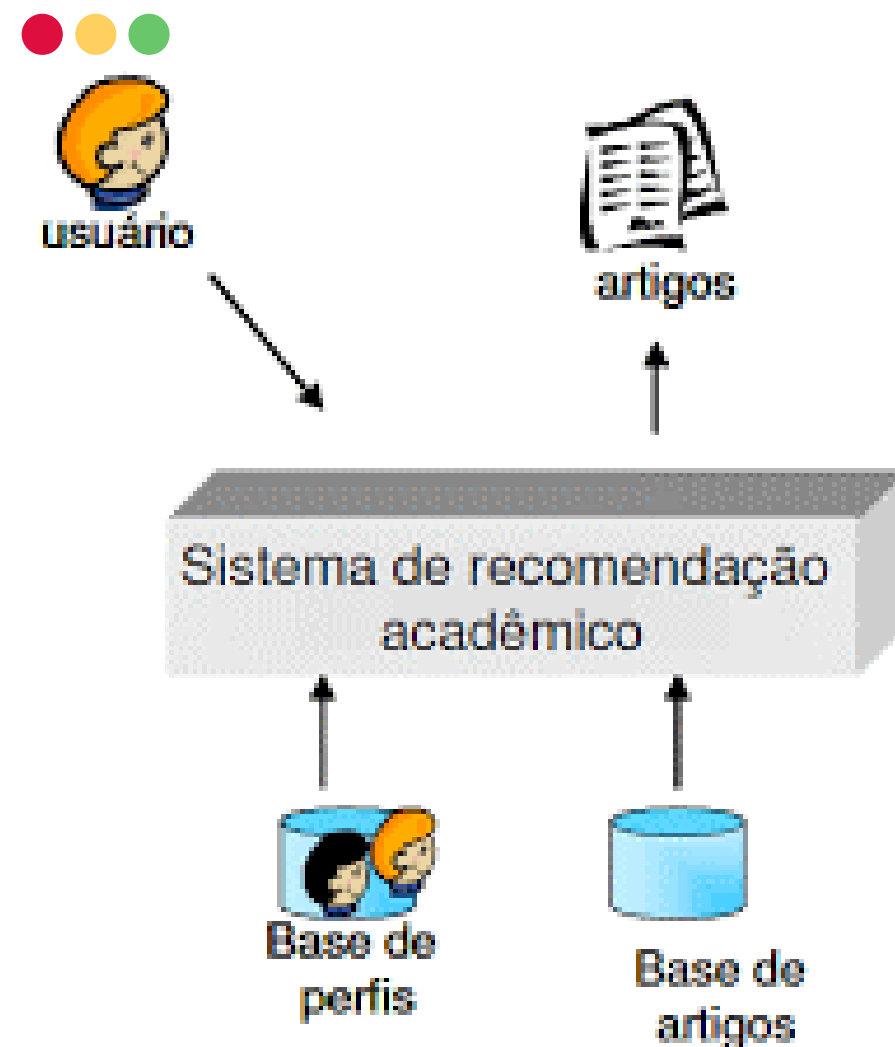


Os dados são armazenados em listas de adjacência implementadas com dicionários Python – uma estrutura escolhida por conta da natureza esparsa do grafo. Com 15 usuários e 30 artigos, a densidade máxima seria de 450 interações, porém o sistema registra apenas 78. Representar isso em uma matriz de adjacência desperdiçaria a maior parte do espaço com zeros. A lista de adjacência aloca memória estritamente para as relações que existem, com complexidade $O(|V| + |E|)$ contra $O(|V|^2)$ da matriz, tornando a estrutura eficiente tanto em memória quanto em tempo de consulta.



**Fila
x
Matriz**

Entradas e Saídas



Entradas

Dados (CSVs):

- Catálogo de artigos acadêmicos (título, área, ano)
- Cadastro de usuários (nome, área de interesse)
- Histórico de interações usuário → artigo

Interação do usuário:

- Usuário existente → informa seu ID
- Usuário novo → escolhe área de interesse

Saídas

Lista ranqueada de artigos recomendados contendo:

- Score de relevância
- Quantidade de conexões com artigos já lidos
- Quais artigos motivaram a recomendação

Entradas e Saídas

```
Escolha uma opção: 1
Digite seu ID de usuário: U04

Olá, Diego Ferreira!
Você leu 5 artigo(s):
[T01] Attention Is All You Need
[T03] A Survey on Transfer Learning
[T06] BERT: Pre-training of Deep Bidirectional Transformers
[T13] Dropout: A Simple Way to Prevent Neural Networks from Overfitting
[T22] Efficient Estimation of Word Representations in Vector Space

=== Recomendações por vizinhos mais próximos ===
1. T26 - Support Vector Machines (Aprendizado de Máquina, 1998)
   score=1.9000 | conexões=4 | maior similaridade=0.5000
2. T18 - Generative Adversarial Networks (Aprendizado de Máquina, 2014)
   score=1.7667 | conexões=4 | maior similaridade=0.6000
3. T05 - Deep Residual Learning (Visão Computacional, 2016)
   score=1.0667 | conexões=4 | maior similaridade=0.4000
```

```
Escolha uma opção: 2
Digite o número da sua área de interesse: 6

Artigos disponíveis em 'Processamento de Linguagem Natural':
[T06] BERT: Pre-training of Deep Bidirectional Transformers (2019)
[T22] Efficient Estimation of Word Representations (2013)

=== Recomendações por vizinhos mais próximos ===
1. T03 - A Survey on Transfer Learning (Aprendizado de Máquina, 2010)
   score=1.4000 | conexões=2 | maior similaridade=1.0000
2. T01 - Attention Is All You Need (Aprendizado de Máquina, 2017)
   score=1.1333 | conexões=2 | maior similaridade=0.8000
3. T13 - Dropout: Preventing Overfitting (Aprendizado de Máquina, 2014)
   score=0.7667 | conexões=2 | maior similaridade=0.6000
```

Análise dos Resultados



Com threshold igual a 0,10, a projeção artigo-artigo gerou:

- 29 nós.
- 92 arestas.
- Grau médio de 6,34.



A projeção filtrada resultou em dois componentes conectados:

- Componente 1: 16 artigos.
- Componente 2: 13 artigos.

O maior componente agrupa principalmente artigos de Aprendizado de Máquina, PLN, Grafos, Sistemas de Recomendação, Visão Computacional e Recuperação de Informação.

O segundo componente reúne principalmente Engenharia de Software, Sistemas Distribuídos e Segurança Computacional.

Limitação:

Um dos artigos, o T19, não foi recomendado por não ter sido lido por nenhum usuário

Análise dos Resultados



Exemplo 1: usuário U01

O usuário U01 possui histórico concentrado em Aprendizado de Máquina, PLN e Visão Computacional. As principais recomendações foram:

T30, T22 e T04 com temas respectivamente:
Visão Computacional, PLN e Recuperação de Informação



Exemplo 2: usuário U08

O usuário U08 tem perfil de Engenharia de Software e Sistemas Distribuídos. As principais recomendações foram:

T27, T02 e T12 com temas respectivamente:
Engenharia de Software, Sistemas Distribuídos e Sistemas Distribuídos

Utilização de LLM's



- **Geração de Dados:** Uso do LLM Claude para criar dados sintéticos de artigos acadêmicos.
- **Armazenamento:** Exportação em formato CSV na pasta /data.
- **Aplicação:** Base de dados utilizada para a construção do sistema de recomendação.

```
├── data/  
│   ├── artigos.csv  
│   ├── usuarios.csv  
│   └── interacoes.csv
```